



KANDIDAT

10213

PRØVE

TDT4186 1 Operativsystemer

Emnekode	TDT4186
Vurderingsform	Skriftlig eksamen
Starttid	29.05.2026 07:00
Sluttid	29.05.2026 11:00
Sensurfrist	19.06.2026 23:59
PDF opprettet	31.05.2026 18:41

Introduction

Oppgave	Tittel	Oppgavetype
i	Copy of Cover Page	Informasjon eller ressurser

Multi-Choice Questions

Oppgave	Tittel	Oppgavetype
1	OS structures	Flervalg
2	Process and Thread	Flervalg
3	Scheduling	Flervalg
4	Memory	Flervalg
5	Paging	Flervalg
6	Concurrency	Flervalg
7	I/O devices	Flervalg
8	Files	Flervalg

Multi-Part Questions

Oppgave	Tittel	Oppgavetype
9	Process	Fyll inn tekst
10	Scheduling	Fyll inn tekst
11	Paging	Fyll inn tekst
12	Concurrency	Fyll inn tekst
13	File systems	Tekstfelt

Comments

Oppgave	Tittel	Oppgavetype
14	Comments	Tekstfelt

1 OS structures

Note: Throughout the exam, a Unix-like operating system is considered.

OS - Operating System

What is the primary role of a bootloader in a computer system?

Select one alternative

- ☐ To manage virtual memory allocation during runtime
- ☒ To load and transfer control to the OS kernel after power-on
- ☐ To handle hardware interrupts and device driver registration
- ☐ To initialize the file system and mount storage partitions

An operating system is designed with the following observed behaviors:

- ① A bug in the file system driver causes it to crash and restart automatically — the rest of the system remains fully operational.
- ② Device drivers and the file system run in user space as separate processes.
- ③ Components communicate by passing messages through a central core, which only handles scheduling, basic IPC, and memory protection.
- ④ Adding a new network protocol requires no changes to the core — a new user-space server is deployed instead.

Based on these characteristics, what type of OS structure does this system use?

Select one alternative

- ☐ Monolithic kernel
- ☐ Hybrid kernel
- ☐ Layer kernel
- ☒ Microkernel

A C program calls `fwrite()` to write data to a file. Internally, the C standard library buffers the data, then eventually calls `write()`. The OS then interacts with the disk driver to flush data to storage.

`write()` is a system call function.

At which point in this sequence does a transition from user space to kernel space occur, and what mechanism triggers it?

Select one alternative

- ☐ When the disk driver accesses hardware registers — direct hardware access forces a privilege escalation from user to kernel mode
- ☒ When `write()` is invoked — it is a system call that triggers a software interrupt (or syscall instruction), causing the CPU to switch to kernel mode
- ☐ When `fwrite()` is called — the C library function immediately switches to kernel mode to access the file descriptor table
- ☐ There is no transition — modern OSes run the entire `fwrite()` call chain in kernel mode for performance

Maks poeng: 4.5

2 Process and Thread

A running process calls `read()` on a file and must wait for the disk. Which state transition sequence correctly describes what happens?

Select one alternative

- ☐ Running → Terminated → Ready, once the disk finishes
- ☐ Running → Ready immediately, then Blocked after the scheduler notices the wait
- ☐ Running stays Running; the OS busy-waits on behalf of the process
- ☒ Running → Blocked (Waiting), then Ready → Running again after the I/O completes

A process calls `fork()` followed immediately by `exec()`. What is the purpose of this two-step pattern, and what happens to the child's original address space?

Select one alternative

- ☐ `fork()` creates a new process sharing the parent's address space; `exec()` makes the memory copy-on-write protected
- ☐ `fork()` suspends the parent; `exec()` resumes it after the child is fully loaded
- ☐ `fork()` creates a new thread inside the same process; `exec()` gives it a separate stack
- ☒ `fork()` duplicates the parent's address space into a new process; `exec()` replaces that address space entirely with a new program image

```
int main() {
    int x = 10;
    int pid = fork();
    if (pid == 0) {
        x = 99;
        printf("child: x = %d\n", x);
    } else {
        printf("parent: x = %d\n", x);
    }
    return 0;
}
```

What does this program print, and why?

Select one alternative

- ☐ Only child: $x = 99$. The parent exits immediately after `fork()` returns
- ☒ child: $x = 99$ and parent: $x = 10$. Each process has its own copy of the address space; the child's write does not affect the parent
- ☐ The output is undefined because both writes race on `stdout`
- ☐ child: $x = 99$ and parent: $x = 99$. `fork()` shares memory, so the child's write is visible in the parent

After `fork()`, a parent immediately calls `wait()`. The child runs for 2 seconds then exits. **What does the parent do during those 2 seconds?**

Select one alternative

- ☐ The parent continues executing concurrently
- ☐ The parent and child take turns executing
- ☐ The parent suspends itself for 2 seconds using an internal timer, then resumes regardless of whether the child has finished
- ☒ The parent blocks until the child exits

Which of the following is shared between two threads of the same process, but NOT shared between two separate processes?

Select one alternative

- ☐ Program counter and CPU register values
- ☐ Thread ID (TID) and Process ID (PID)
- ☐ Stack and parent PID
- ☒ Heap memory and open file descriptors

A server application must handle thousands of concurrent clients. A developer considers two designs: spawning a new process per client, or spawning a new thread per client. **Which statement best explains why threads are generally preferred for this workload?**

Select one alternative

- ☒ Creating a thread is cheaper than forking a process because threads share the address space and require no new page table or file descriptor table duplication
- ☐ Threads bypass the scheduler entirely and are therefore always faster than processes
- ☐ Processes require kernel-mode execution for all operations, while threads stay entirely in user space
- ☐ Threads can run on multiple CPUs simultaneously while processes cannot

Maks poeng: 9

3 Scheduling

Which of the following claims about FIFO, SJF, and STCF is correct?

Select one alternative

- ☒ SJF improves turnaround time over FIFO, but a short job that arrives after a long job has already started must still wait for it to finish
- ☐ FIFO minimises average turnaround time because jobs are processed in the order they arrive, avoiding any scheduling overhead
- ☐ STCF improves response time over SJF by preempting the current job whenever a shorter one arrives
- ☐ FIFO and SJF produce identical turnaround times as long as all jobs arrive at the same time

Which of the following statements about Round Robin scheduling is true?

Select one alternative

- ☒ RR improves response time but may hurt turnaround time
- ☐ RR improves both response time and turnaround time compared to FIFO
- ☐ RR improves turnaround time but may hurt response time
- ☐ RR and FIFO produce identical turnaround times

Note: The runnable state is equivalent to the ready state.

Two processes P1 and P2 have equal weights and are both runnable. CFS has been running them for a while. At a scheduling decision point, P1 has accumulated more virtual runtime than P2.

Which process does CFS select next, and what property of CFS does this demonstrate?

Select one alternative

- ☐ P1. CFS rewards processes that have been waiting longer by giving them extra CPU time to catch up
- ☐ Either P1 or P2. CFS selects randomly among runnable processes to ensure fairness
- ☐ P1. CFS prioritises processes that have already used more CPU to ensure they finish sooner
- ☒ P2. CFS always selects the process with the least virtual runtime, ensuring each process receives a proportionally equal share of the CPU over time

Building on the previous question, now P1 has twice the weight of P2. Both are runnable. CFS tracks virtual runtime as:

$$vruntime_i = vruntime_i + \frac{weight_0}{weight_i} \cdot runtime_i$$

Over time, which of the following best describes how CFS allocates CPU between P1 and P2?

Select one alternative

- ☒ P1 receives twice as much actual CPU time as P2
- ☐ P1 receives all CPU time until its virtual runtime exceeds P2's, then they alternate equally
- ☐ P1 and P2 still receive equal CPU time
- ☐ P2 receives twice as much actual CPU time

Maks poeng: 6

4 Memory

Two processes are both running on the same machine. Process A uses virtual address $0x1000$ to store its variable x . Process B also uses virtual address $0x1000$ to store its variable y . **Why do these two processes not interfere with each other?**

Select one alternative

- ☐ The OS ensures that no two processes ever use the same virtual address by assigning each process a unique address range
- ☐ The CPU detects address conflicts at runtime and automatically redirects one process to a different address
- ☐ Processes run one at a time, so only one process ever uses address $0x1000$ at any given moment
- ☒ Virtual memory gives each process its own private address space, so the same virtual address in two different processes maps to different physical locations

A programmer is debugging a program and finds a bug at address $0x1000$. She tries to inspect physical address $0x1000$ directly using a hardware debugger, expecting to find the faulty data there. **Why does this not work as expected?**

Select one alternative

- ☐ Physical address $0x1000$ belongs to a different process, so the OS would need to context switch before the data becomes visible
- ☒ The address $0x1000$ the programmer sees is a virtual address, so the faulty data does not reside at physical address $0x1000$
- ☐ Physical addresses are only used for heap and stack memory, and code and global variables are always accessed through virtual addresses, so inspecting a physical address will never reveal a code bug
- ☐ Physical memory is shared between all running processes, so inspecting physical address $0x1000$ cannot identify which process's virtual address $0x1000$ is being debugged

A program is compiled assuming its virtual addresses start at $0x0$. At runtime, the OS loads it into physical address starting at address $0x4000$. Dynamic relocation uses a base register to handle this. **What does dynamic relocation do?**

Select one alternative

- ☐ The base register is set once at compile time and never changes
- ☐ The base register stores the size of the process
- ☐ The base register maps each virtual address independently to any physical location
- ☒ The base register shifts every virtual address by a fixed offset to produce a physical address

A system uses segmentation. The OS reports that 8 MB of physical memory is free. A process requests a 6 MB contiguous segment and the request fails. **Which of the following best explains why?**

Select one alternative

- ☒ The free memory is scattered across multiple non-contiguous holes
- ☐ The request fails because segmentation causes internal fragmentation
- ☐ Segmentation limits each segment to a fixed maximum size, and 6 MB exceeds that limit
- ☐ The request fails because 6 MB exceeds the maximum segment size imposed by the bounds register

Maks poeng: 6

5 Paging

A system uses segmentation to manage memory. Over time, the OS struggles to satisfy memory requests even though enough total free memory exists. A new OS design switches to paging.

Which of the following best explains what fundamental problem paging solves that segmentation does not?

Select one alternative

- ☒ Segmentation allocates variable-size regions, leading to external fragmentation, while paging uses fixed-size pages so free memory never becomes unusably scattered
- ☐ Segmentation loads the entire program into memory at once, while paging allows parts of a program to remain on disk
- ☐ Segmentation allows processes to access each other's memory, while paging enforces strict isolation between processes
- ☐ Segmentation requires a separate base and bounds register per segment, while paging eliminates the need for any hardware support for address translation

After calling `fork()`, a developer notices that the child process starts almost instantly and system memory usage barely increases — even though the parent has 2 GB of heap data. Unmodified pages appear to be shared between parent and child, but when the child modifies a variable, the parent's copy is unaffected. **What mechanism explains this?**

Select one alternative

- ☒ Copy-on-write
- ☐ Demand paging
- ☐ Swapping
- ☐ Memory-mapped files

In xv6-riscv, a 39-bit virtual address is structured as follows:

| VPN[2] (9 bits) | VPN[1] (9 bits) | VPN[0] (9 bits) | offset (12 bits) |

The OS walks three levels of page tables to translate a virtual address to a physical one. **Which of the following best describes the role of the 12-bit offset?**

Select one alternative

- ☒ The offset identifies the exact byte location within a physical page
- ☐ The offset indexes into the top-level page table to find the physical frame number
- ☐ The offset selects which page table level to use
- ☐ The offset is the difference between the virtual page number and the physical frame number, used to correct the translation

In xv6-riscv, address translation uses a three-level page table. Assume the required translation is not in the TLB, so the hardware must walk the page table in memory. To perform one data read from a virtual address, **how many total memory accesses are required, including both page-table walks and the final data read?**

Select one alternative

- ☐ Two accesses
- ☐ One access
- ☐ Three accesses
- ☒ Four accesses

An OS supports two page sizes: 4KB pages for normal processes and 2MB pages for database applications that manage large datasets. A developer needs to determine the offset bits for each page size to understand how virtual addresses are split. **Which of the following correctly derives both?**

Select one alternative

- ☐ Both use 12 offset bits
- ☐ Normal pages use 12 offset bits and large pages use 24 offset bits
- ☒ Normal pages use 12 offset bits and large pages use 21 offset bits
- ☐ Normal pages use 10 offset bits and large pages use 21 offset bits

A database process accesses pages in a completely random order, that is, each page access is independent with no predictable pattern. The OS designer must choose between demand paging and prefetching for this workload. **Which strategy is more suitable and why?**

Select one alternative

- ☐ Prefetching is more suitable. Random access causes frequent page faults under demand paging, and prefetching eliminates all of them by loading the entire working set upfront
- ☐ Prefetching is more suitable. Even with random access, loading pages ahead of time reduces the total number of page faults regardless of access pattern
- ☐ Both strategies perform identically on random workloads. The access pattern only affects which pages are loaded, not when they are loaded
- ☒ Demand paging is more suitable. Since access patterns are random, prefetching predictions will frequently be wrong, wasting memory and disk I/O on pages that are never accessed

A system has 3 physical frames, initially empty. A process accesses pages in the following order:
1, 2, 3, 1, 4, 1, 2

How many page faults occur under FIFO replacement?

Select one alternative

- ☐ 5
- ☐ 3
- ☐ 4
- ☒ 6

A system has 4 processes each requiring 256MB of RAM, but the machine only has 512MB of physical memory. All four processes are running. **Why is the OS able to run all four processes simultaneously despite the memory shortage?**

Select one alternative

- ☒ The OS swaps pages of inactive processes out to disk, freeing physical frames
- ☐ The OS gives each process 128MB, and each process manages its own memory shortage
- ☐ The OS compresses inactive processes' memory so all four fit within 512MB
- ☐ The OS serialises execution so only two processes run at a time, keeping memory usage within 512MB

A system uses the Clock algorithm (the second hand, or the practical LRU implementation) for page replacement. Four physical frames contain pages A, B, C, D with the following reference bits:

Frame Page Reference bit

0	A	1
---	---	---

Frame Page Reference bit

1	B	0
2	C	1
3	D	0

The clock hand is currently pointing at frame 0 (page A). A page fault occurs and a new page must be loaded. **Which page gets evicted?** (Reference bit indicates whether the page is used recently.)

Select one alternative☒ B☐ C☐ A☐ D

Maks poeng: 13.5

6 Concurrency

A producer-consumer program uses **a lock** and **a condition variable**. The consumer is implemented as follows:

```
lock(&mutex);  
if (count == 0) {  
    wait(&cond, &mutex);  
}  
consume();  
unlock(&mutex);
```

A bug occasionally causes `consume()` to run even when the buffer is empty. **How to fix it?**

Select one alternative

- ☐ Move `wait()` outside the lock so the producer can signal without contention
- ☒ Replace `if` with `while` so the condition is rechecked after the thread wakes up
- ☐ Move `consume()` outside the critical section so it does not need to hold the lock
- ☐ Remove the lock entirely and rely on the condition variable for mutual exclusion

Two threads acquire two locks in opposite orders:

Thread 1: `lock(A) → lock(B)`

Thread 2: `lock(B) → lock(A)`

Thread 1 holds lock A and waits for lock B. Thread 2 holds lock B and waits for lock A. A developer fixes the deadlock by enforcing that all threads must always acquire lock A before lock B. **Which of the four necessary conditions for deadlock does this fix break?**

Select one alternative

- ☐ No preemption
- ☐ Mutual exclusion
- ☐ Hold and wait
- ☒ Circular wait

A thread tries to acquire a lock using a spin loop:

```
while (lock_is_held()) {  
    // spin  
}  
lock(&mutex);  
// critical section  
unlock(&mutex);
```

The lock is currently held by another thread that is waiting for a disk I/O operation expected to take 200ms. **Which of the following statements about this approach is true?**

Select one alternative

- ☐ The spin loop will cause a deadlock
- ☒ The spinning thread wastes CPU cycles for 200ms doing no useful work
- ☐ The spin loop will never terminate
- ☐ Spinning is appropriate here

Two threads must execute in a specific order — thread B must always run after thread A completes a certain operation. The following code is used:

```
sem_t sem;
sem_init(&sem, ?, ?); // what should ? be?
// Thread A
do_work();
sem_post(&sem);
// Thread B
sem_wait(&sem);
do_next();
```

What should the initial value of the semaphore be?

Select one alternative

- ☐ 1
- ☐ 2
- ☐ -1
- ☒ 0

Maks poeng: 6

7 I/O devices

A system has two devices: a fast network card that completes operations in 1 microsecond, and a slow disk that completes operations in 10ms. The OS designer must choose between polling and interrupts for each device. **What is the most appropriate I/O strategy for each device?**

Select one alternative

- ☐ Use interrupts for the network card and polling for the disk
- ☐ Use polling for both devices
- ☒ Use polling for the network card and interrupts for the disk
- ☐ Use interrupts for both devices

A developer notices that writing to memory address `0xFFFF0000` controls a device, not actual RAM. **What causes this behavior?**

Select one alternative

- ☐ The CPU automatically routes writes to this address range to the device controller via I/O instructions
- ☐ The OS remapped the device's physical address to protect it from user processes
- ☒ Device registers are mapped into the physical address space
- ☐ The compiler redirects writes to unused memory addresses toward connected devices

A process requests a read of a 100MB file from disk. The OS sets up a DMA transfer by providing the disk controller with the source, destination, and size. **What does the CPU do during the transfer?**

Select one alternative

- ☐ The CPU continuously updates the DMA controller with the next memory address to write to as the transfer progresses
- ☐ The CPU suspends all processes and waits for the DMA controller to finish
- ☐ The CPU polls the DMA controller's status register until it reports completion
- ☒ The CPU is free to execute other processes

A system needs to transfer a large file from disk to memory. The OS initiates the transfer by writing the source address, destination address, and transfer size to device registers mapped to specific memory addresses. The disk controller then takes over the transfer independently. When the transfer completes, the device notifies the CPU.

Which of the following correctly identifies the three mechanisms being used in this scenario?

Select one alternative

- ☒ Memory-mapped I/O for initiation, DMA for the transfer, and an interrupt for completion
- ☐ Memory-mapped I/O for initiation, polling for the transfer, and an interrupt for completion
- ☐ Polling for initiation, PIO for the transfer, and an interrupt for completion
- ☐ I/O instructions for initiation, DMA for the transfer, and polling for completion

Maks poeng: 6

8 Files

A process calls `open()` on the same file twice:

```
int fd1 = open("file.txt", O_RDONLY);  
int fd2 = open("file.txt", O_RDONLY);
```

Which of the following is TRUE?

Select one alternative

- ☐ The second `open()` fails and returns -1
- ☐ fd1 and fd2 share the same offset
- ☐ Both fd1 and fd2 refer to the same file descriptor
- ☒ fd1 and fd2 are two distinct file descriptors

Given only an inode number, what information can the OS retrieve about a file?

Select one alternative

- ☒ The file's metadata (size, permissions, timestamps) and pointers to its data blocks, but not its filename
- ☐ Only the file's size and owner
- ☐ The full path of the file and the directory entry it belongs to
- ☐ The filename, its location in the directory tree, and the data block contents

A process calls `unlink("file.txt")`. The file's inode has a reference count of 2. **What happens to the path and the inode?**

Select one alternative

- ☐ The path and the inode are both kept
- ☐ The path is removed from the directory and the reference count is decremented to 0
- ☒ The path is removed from the directory and the reference count is decremented to 1
- ☐ The path is removed from the directory and the inode is freed immediately

A simple file system contains an inode bitmap and a data block bitmap. **What is the role of each?**

Select one alternative

- ☐ The inode bitmap maps filenames to inode numbers, and the data block bitmap maps inode numbers to data block addresses
- ☒ The inode bitmap tracks which inodes are free or allocated, and the data block bitmap tracks which data blocks are free or allocated
- ☐ The inode bitmap only tracks inodes used by the current process, and the data block bitmap only tracks blocks allocated in the current session
- ☐ The inode bitmap stores the file permissions and the data block bitmap stores the file sizes

A process renames a file from `old.txt` to `new.txt`. **Which file system structures are modified?**

Select one alternative

- ☐ A new inode is allocated for `new.txt` and the old inode is freed
- ☐ Both the inode and path are modified
- ☐ The inode is updated to store the new filename, and the path is updated to point to the new inode
- ☒ The old path is removed and a new path is created pointing to the same inode, while the inode remains untouched

A process creates a new file `file.txt` in `/home/user/`. **Which metadata structure associated with `/home/user/` is updated after creating `file.txt`?**

Select one alternative

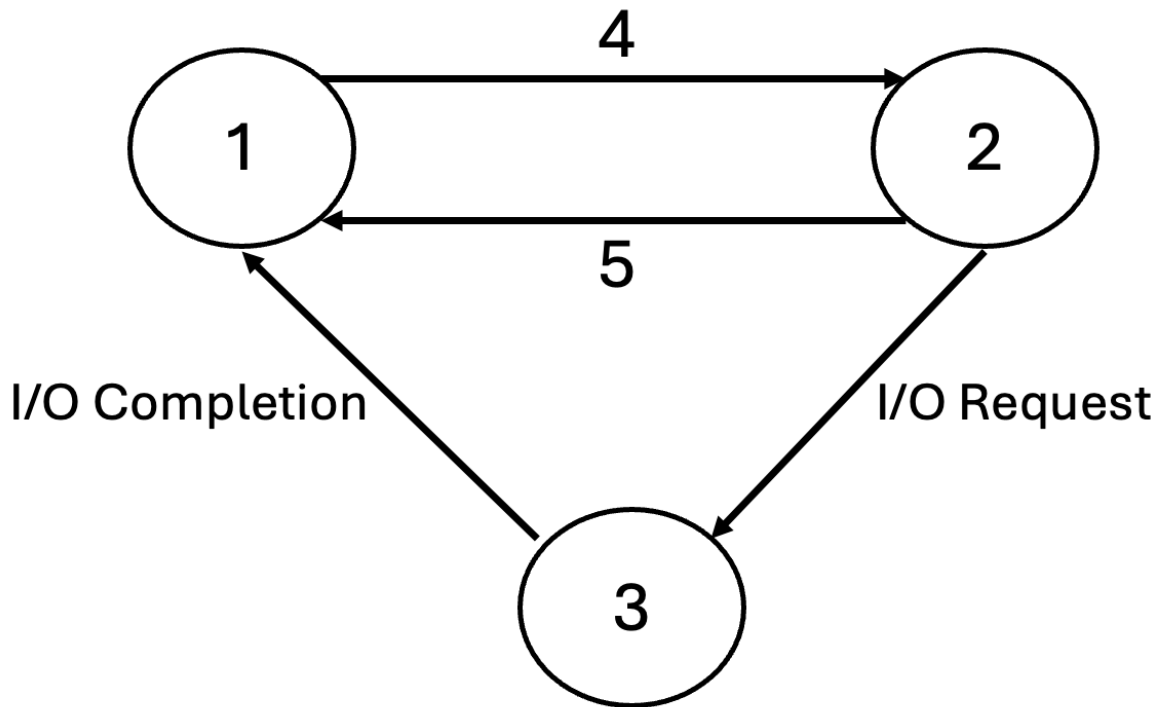
- ☐ The data bitmap
- ☐ The parent directory's inode (user)
- ☒ The data block
- ☐ The inode bitmap

Maks poeng: 9

9 Process

Each blank is worth 1 point.

The following diagram shows the state transition graph of a process. States 1, 2, and 3 represent the three main process states. Transitions 4 and 5 are unlabelled, and the state names are missing.



Fill in the blanks:

1.
2.
3.
4.
5.

Maks poeng: 5

10

Scheduling

Each blank is worth 2 points.

A system uses an MLFQ scheduler with the following configuration:

Queue	Priority	Scheduling Policy	Time Slice
Q0	Highest	Round Robin	4 ms
Q1	Medium	Round Robin	8 ms
Q2	Lowest	FIFO	Runs to completion

At each queue, the **time allotment** is **two time slices**.

When a new process enters the system, it is placed in queue

After a process executes 10ms, it is placed in queue

The system changes to a CFS scheduler. Note: in Linux CFS, weights are derived from nice values and the default weight is 1024. For simplicity, this question uses simplified weights that do not reflect actual Linux values. In this question, a higher weight means higher priority, and the default weight is **1**. Two processes are scheduled in the system.

Process	Weight	Arrival Time
P1	3	0ms
P2	1	0ms

The scheduling period is 40ms.

P1's time slice = 30ms

P2's time slice = ms

$$time_slice_k = \frac{weight_k}{\sum_{i=0}^{n-1} weight_i} \cdot sched_latency$$

After one scheduling period,

P1 accumulates ms virtual runtime.

P2 accumulates 10 ms virtual runtime.

$$vruntime_i = vruntime_i + \frac{weight_0}{weight_i} \cdot runtime_i$$

In the next scheduling period, a third process P3 with weight 2 joins. The total weight is now 6.

P3 receives CPU time than P2 because it has a higher weight.

11 Paging

Each blank is worth 2 points.

A system uses paging for memory management with the following configuration:

- Virtual address: 32 bits
- Page size: 4KB
- Each page table entry: 4 bytes
- TLB size: 4 entries
- Physical memory: 4 frames (for replacement policy questions)

How many bits are used for page index? bits

If the system uses a single-level page table, it may waste memory for unused regions. Thus, a two-level page table is used. How many bits are used for each page directory level in the two-level page table? bits

The virtual address $0x12345678$ is divided into:

- Page directory index = $0x$ (The first/top level)
- Page table index = $0x$ (The second level)

A process accesses the following virtual addresses in order:

Access	Virtual Address
1	0x00401ABC
2	0x00802DEF
3	0x00401F23
4	0x00C03456
5	0x00802B89

The TLB is initially empty. How many TLB misses occur for this access sequence?

NOTE: TLB stores the full virtual-to-physical mapping.

Maks poeng: 10

12 Concurrency

Each blank is worth 1 point.

Three processes P_1 , P_2 , and P_3 share a bounded buffer containing N ($N > 0$) slots.

- P_1 repeatedly generates a positive integer using `produce()` and places it into an empty buffer slot using `put()`
- P_2 repeatedly retrieves an odd number from the buffer using `getodd()` and counts it using `countodd()`
- P_3 repeatedly retrieves an even number from the buffer using `geteven()` and counts it using `counteven()`

The three processes must operate with the following constraints:

- The buffer has a maximum capacity of N slots
- Only one process may access the buffer at a time
- P_2 must wait if there are no odd numbers in the buffer
- P_3 must wait if there are no even numbers in the buffer

Please fill in blanks to complete all processes.

semaphore empty = N ;

semaphore even = ;

semaphore odd = ;

semaphore mutex = ;

P1:

```
while(1) {
  x = produce();
  sem_wait(&empty);
  sem_wait(&mutex);
  put(x);
  sem_post(&mutex);
  if (x % 2 == 0)
```

else

}

P2:

```
while(1) {
```

```
sem_wait(&mutex);
getodd();
```

```
sem_post(&mutex);  
countodd();  
sem_post(&empty);  
}
```

P3:

```
while(1) {
```

```
    sem_wait(&even)
```

```
    sem_wait(&mutex);  
    geteven();  
    sem_post(&mutex);  
    counteven();  
    sem_post(&empty);  
}
```

Maks poeng: 7

13 File systems

Each blank is worth 2 points.

Suppose the process opens:

/home/user/report.txt

List the sequence of file system objects accessed to locate the file:

1. Root directory inode
2. **Write your answer in the box below. Changes are saved automatically.**
root directory datablock
3. home directory inode
4. **Write your answer in the box below. Changes are saved automatically.**
home directory datablock
5. user directory inode
6. **Write your answer in the box below. Changes are saved automatically.**
user directory datablock
7. **Write your answer in the box below. Changes are saved automatically.**
report.txt inode

Maks poeng: 8

14 Comments

You can write some comments for your solutions here; these will not be graded.

Fill in your answer here

Ignored semicolons for question 12

Did not include leading 0 for hex number on question 11

Assumed last part of question 10 asked for more/less/same and not what multiple of cpu time it should be

Maks poeng: 0